

Technical Implementation Proposal

Engineer-facing system contract for property publishing, demand capture, CRM, billing, integrations, AI controls, security, observability, and acceptance.

Prepared by Nimbuz Tech

13 August 2026

Companion to the executive sales proposal

Eens Limited · eensbpark.ke

Eens Limited

Technical Implementation Proposal

Companion to: eens-executive-sales-proposal.md **Prepared by:** Nimbuz Tech **Client:** Eens Limited
Operational site: eensbpark.ke **Document reference:** NT-PRP-2026-EENS-TECH **Date:** 13 August 2026
Status: Proposed implementation contract; verify configuration during Phase 0

1. Purpose and boundary

This document is the engineer-facing companion to the executive sales proposal. It defines how the installed Eens stack should be connected without presenting proposed integrations as live.

The implementation target is:

Public demand

-> Astro/Tina property publishing

-> validated enquiry or conversation

-> n8n orchestration and attribution

-> Frappe CRM Lead / Deal

-> Utility Service Request and approved property match

-> Contract, deposit, rent, meter billing, invoice, payment

-> reporting, renewals, content feedback, and permission-controlled assistant analysis

This document covers:

- system ownership and source-of-truth boundaries;
- verified applications, DocTypes, and current gaps;
- proposed public projections, API methods, and event contracts;
- n8n, channel, payment, analytics, and CMS integration flows;
- Assistant Core skills, tools, permissions, and approval gates;
- security, privacy, idempotency, observability, and recovery;
- vertical-slice delivery and acceptance tests.

It does not authorize production credentials, payment activation, destructive CMS writes, or unattended financial actions.

2. Verified current stack

The eensbpark.ke Bench and checked source were audited on 13 August 2026.

Component	Verified state	Engineering responsibility
Frappe Framework 16.25.0	Installed and running	DocTypes, controllers, roles, permissions, REST methods, hooks, jobs, files, notifications, and Desk

Component	Verified state	Engineering responsibility
ERPNext 16.26.2	Installed and running	Customer, item, order, invoice, payment, account, asset, project, report, and accounting lifecycle
CRM 1.81.1	Installed and running	Lead, deal, task, source, status, territory, owner, calls, notes, SLA, and handoff
Utility & Rental Billing 0.0.1	Installed and running	Utility Property, service requests, contract-unit links, meters, tariffs, adjustments, recurring billing, and reports
Assistant Core 2.5.0	Installed and running	Permission-scoped tools, prompt templates, skills, dashboards, analysis, code-execution controls, and audit logs
Communication layer	Installed and running	Profiles, topics, messages, templates, channel routing, document links, notifications, and webhooks
eens_app	Installed; no custom business DocTypes or active business hooks	Eens-owned integration APIs, projections, skills, and domain rules
Astro 7.2 + TinaCMS	Present under eens_app/frontend	Static property publishing, editorial content, routes, metadata, sitemap, manifest, and service-worker shell

Current gaps that must remain explicit

The inspected source does not evidence:

- an API synchronisation between ERP property records and Tina records;
- a public lead endpoint creating CRM records;
- an installed n8n runtime in the Bench;
- a payment-provider integration or reconciliation handler;
- public analytics event capture and source-to-revenue joins;
- configured public channel credentials and templates;
- Eens-specific Assistant Core tools or skills;
- custom Eens DocTypes, scheduled jobs, or business hooks.

The implementation must add these incrementally and test each boundary before enabling it.

3. Ownership and source of truth

The system needs two records of responsibility and one controlled integration contract.

Concern	System owner	Rule
Unit identity, occupancy, contract state, meter, invoice, payment, and internal status	Frappe/ERPNext + Utility Billing	Operational truth; never expose the full record publicly
Public copy, page composition, SEO fields, media, guides, and editorial review	TinaCMS/Astro	Publishing truth; only approved fields are projected from operations
Enquiry and commercial follow-through	CRM	Lead and Deal own source, owner, SLA, next action, and commercial status

Concern	System owner	Rule
Conversation delivery and channel context	Communication layer	Link conversations to the CRM or operational record; do not replace CRM
Workflow execution state	n8n	Orchestrates validated events; never becomes business truth
Assistant context and action policy	Assistant Core + eens_app extensions	Permission-scoped; read-first; audited writes only
Accounting and receivables	ERPNext	Financial submission and reconciliation remain controlled by finance roles

Transition rule

Until a public projection and human review workflow are accepted, the public register remains manually reviewed. Do not introduce direct destructive CMS mutations or browser calls to private ERP records.

4. Current DocType map

4.1 Frappe and ERPNext foundation

Use standard records before creating custom records:

- User, Role, Role Profile, DocType, File, Communication, Workflow, Notification, and Webhook;
- Company, Customer, Contact, Address, Territory, and User;
- Item, Item Price, Sales Order, Sales Invoice, Payment Entry, Account, Cost Center, Project, and Auto Repeat;
- Task, Issue, Assignment Rule, and standard reports where they fit the operating workflow.

4.2 CRM records

The checked CRM app provides the commercial intake layer:

- CRM Lead for person/organisation, contact, source, territory, status, owner, SLA, and requirement context;
- CRM Deal for opportunity value, probability, expected closure, next step, source, owner, and outcome;
- CRM Task for the next owned action, due date, status, priority, and linked record;
- lead source, lead status, territory, industry, organisation, call log, notes, communication, and settings records.

A website, channel, or campaign event should create or update one CRM Lead. A Utility Service Request should begin only after the business requirement is sufficiently qualified.

4.3 Utility and rental records

The utility app provides the property-to-cash model:

- Utility Property for estate, building, floor, unit, status, type, location, size, and property facts;
- Utility Service Request for a prospect/customer request, service requirements, property selection, dates, terms, and fulfilment state;

- **Contract Utility Property Item** for the contract-to-unit relationship, active state, dates, contract length, adjustment rule, and insurance link;
- **Utility Bill Structure** for reusable billable items and monthly amounts;
- **Meter Reading** for customer, property, date, meter number, previous/current reading, consumption, utility items, price list, company, cost center, and project;
- **Billing Adjustment Rule** for increments, frequency, grace periods, penalties, caps, accounts, and submission behaviour;
- **Utility Billing Settings** for customer creation, site survey, deposit gates, contract gates, invoice state, merged invoices, and tenancy-ending notice period.

4.4 Assistant records

Assistant Core provides configuration and audit boundaries for:

- settings and connection policy;
- prompt templates and categories;
- skills, tool configurations, plugin configurations, and dashboards;
- assistant audit entries containing user, tool, target, status, timing, input/output, errors, and trace information.

The Eens app should add only narrow domain tools and skills through the supported extension surfaces.

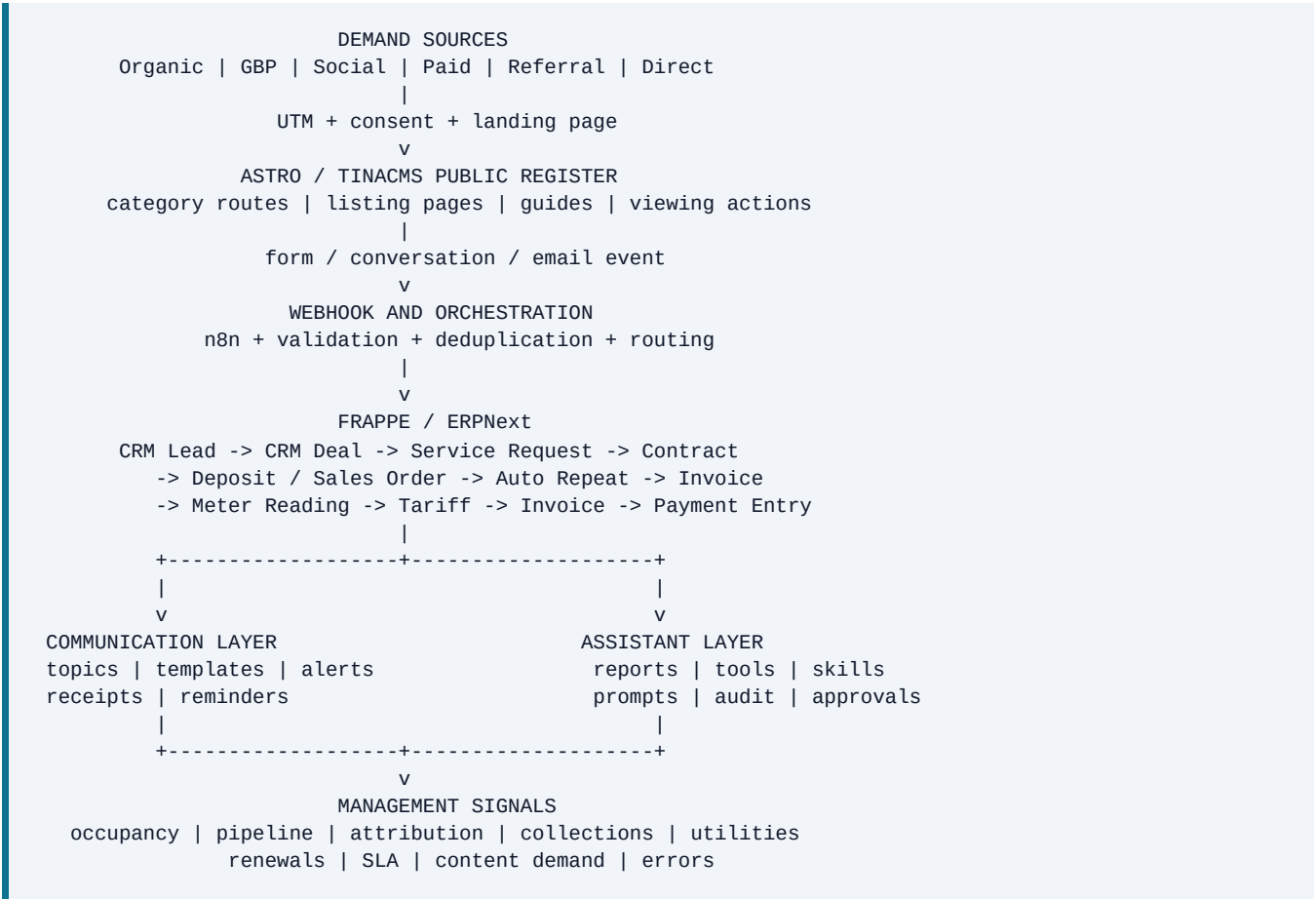
4.5 Public content records

Tina property content currently carries type, availability, address, zone, area, price, terms, references, media, occupancy state, review information, and industrial specifications. The integration should add or project only the minimum operational fields needed for public trust:

- stable listing reference;
- approved publication state;
- availability state;
- last reviewed date and reviewer;
- public terms and approved specifications;
- public page status and source version.

Private tenant names, contacts, balances, internal notes, contract files, credentials, and workflow commentary must never enter the public projection.

5. Target architecture



Public rendering rule

Astro pages must remain statically renderable and must not depend on live ERP or assistant calls in the critical page path. Public pages consume approved editorial content or a controlled build-time projection. Private operational records remain behind authenticated Frappe interfaces.

6. Integration contracts

6.1 Public listing projection

Expose a versioned allowlist, not a full DocType:

```
{
  "reference": "EE-ML0-014",
  "title": "Mlolongo Warehouse - Unit 02",
  "type": "WAREHOUSE",
  "zone": "Mlolongo",
  "address": "Mlolongo, Mombasa Road, KM 14",
  "availability": "ForRent",
  "occupancyState": "available",
  "sqft": "9,000 sq ft",
  "price": { "ksh": "3,150,000 / yr", "perSqft": "35" },
  "leaseTerm": "3-year minimum",
  "specSheet": {
    "power": "150",
    "water": "8",
    "parking": 6,
    "floorLoading": "30",
    "clearHeight": "7.5"
  },
  "lastReviewedDate": "2026-08-13",
  "sourceVersion": 3
}
```

The example values are contract fixtures, not live inventory claims. Production values must come from approved records.

6.2 Proposed Frappe methods

Use standard Frappe REST resources where possible. Add whitelisted methods only for domain operations that need validation, allowlists, or idempotency.

Method	Purpose	Required controls
GET /api/method/eens_app.api.public_listings.list	Paginated approved listing projections	Read-only, cacheable, no private fields
GET /api/method/eens_app.api.public_listings.get	One approved listing projection	Validate reference and publication state
POST /api/method/eens_app.api.leads.capture	Create or update a CRM Lead	Consent, payload limits, source, idempotency key
POST /api/method/eens_app.api.viewings.request	Create a controlled viewing task/request	Listing reference, contact, preferred window
POST /api/method/eens_app.api.webhooks.receive	Receive provider or n8n events	Signature, timestamp, source, replay key
GET /api/method/eens_app.api.health	Report integration health	No secrets or internal traces

All responses need a consistent error envelope. External payloads must be schema-validated before they enter Frappe.

6.3 Event envelope

```
{
  "eventId": "provider-or-workflow-id",
  "eventType": "lead.captured",
  "occurredAt": "2026-08-13T10:30:00Z",
  "source": "website",
  "correlationId": "request-id",
  "idempotencyKey": "sha256-of-source-and-event",
  "payload": {
    "name": "Prospect name",
    "email": "prospect@example.com",
    "phone": "redacted-in-logs",
    "listingReference": "EE-ML0-014",
    "zone": "Mlolongo",
    "requirement": "9,000 sq ft warehouse",
    "consent": true,
    "utm": {
      "source": "google",
      "medium": "organic",
      "campaign": "mlolongo-warehouse"
    }
  }
}
```

Handlers must be retry-safe. Replaying an event must update or acknowledge the existing record rather than create a duplicate lead, message, contract, invoice, or payment.

7. n8n orchestration design

n8n is an integration coordinator. It validates, enriches, routes, retries, and records execution metadata. It does not own property availability, accounting truth, or CRM lifecycle.

Workflow A — approved inventory publishing

```
Frappe availability/property change
-> webhook or scheduled export
-> validate public fields and review state
-> create publishing task or approved content payload
-> human review in the editorial path
-> rebuild Astro
-> verify page, canonical, metadata, and sitemap
-> record publication event and listing reference
```

Workflow B — lead capture

```
Website / search ad / social / conversation
-> validate fields and consent
-> attach UTM, landing page, source, campaign, timestamp
-> deduplicate by identity and recent activity
-> create/update CRM Lead
-> assign owner and SLA
-> acknowledge and notify the responsible team
```

Workflow C — qualification and viewing

CRM Lead or conversation

- > capture category, zone, size, use, power, budget, timeline
- > match approved inventory
- > create CRM Task/Deal when qualified
- > arrange viewing
- > link property references and approved transcript/file links
- > escalate if response target is missed

Workflow D — contract and billing handoff

Qualified CRM Deal

- > Utility Service Request
- > selected Utility Property unit(s)
- > survey/fulfilment checklist where required
- > deposit Sales Order
- > approval and deposit confirmation
- > Contract Utility Property Item
- > Auto Repeat and invoice schedule

Workflow E — billing and payment reconciliation

Meter Reading or due Auto Repeat

- > calculate usage/rent
- > create Sales Order/Sales Invoice according to settings
- > send controlled statement/notice
- > receive approved provider event
- > verify signature, reference, timestamp, and idempotency
- > create Payment Entry or finance exception task
- > notify and record outcome

Workflow controls

Every workflow requires:

- source event ID and idempotency key;
- input validation and payload limits;
- bounded retries and a visible exception/dead-letter path;
- correlation ID across provider, n8n, Frappe, and message events;
- redacted logs and no secrets in execution data;
- owner, escalation, and manual fallback;
- approval before public availability, financial submission, or outbound campaign action.

8. Analytics and data contract

Capture fields

At minimum, capture:

- source, medium, campaign, landing page, listing reference, and first-touch timestamp;
- consent state, contact details, requirement, location, size, use, budget band, and timeline;

- owner, response target, status, last response, next action, viewing date, outcome, offer state, and lost reason;
- customer, service request, contract, invoice, payment, and renewal references after conversion.

Event names

Use stable, versioned names such as:

- `listing.viewed.v1`;
- `listing.enquiry_submitted.v1`;
- `lead.created.v1`;
- `lead.qualified.v1`;
- `viewing.requested.v1`;
- `viewing.completed.v1`;
- `deal.created.v1`;
- `contract.created.v1`;
- `invoice.submitted.v1`;
- `payment.reconciled.v1`;
- `renewal.action_due.v1`.

Do not send tenant identity, balances, payment credentials, or private notes to an advertising destination. Join source-to-revenue inside the controlled operational reporting layer.

Management dashboards

- Availability and listing freshness.
- Lead and deal pipeline, response time, SLA breaches, and viewing conversion.
- Channel and campaign-to-qualified-conversion performance.
- Receivables, ageing, deposits, and payment exceptions.
- Contracts, renewals, escalation dates, and notices.
- Meter readings, consumption, tariff blocks, and abnormal usage.
- Content landing-page demand, unanswered questions, and stale records.

9. AI integration and governance

Initial mode: read, explain, compare, prepare

Approved users should be able to ask:

- Which approved units match a location and size requirement?
- Which listings have not been reviewed recently?
- Which leads have no next action?
- Summarise viewings by source and outcome.
- Which contracts end in the next six months?
- Compare utility consumption across periods.
- Prepare a collections follow-up list grouped by owner and ageing.

- Extract obligations and dates from an approved lease or inspection document.

The assistant should prefer existing ERPNext and Utility Billing reports before custom analysis. Custom analysis must be bounded and read-only unless a separate action is approved.

Proposed Eens skills

Register these as Eens-owned skills after the base contracts are tested:

- `property_availability_review;`
- `lead_followup_review;`
- `viewing_pipeline_summary;`
- `tenancy_renewal_review;`
- `utility_consumption_review;`
- `content_opportunity_review;`
- `monthly_management_pack .`

Proposed narrow tools

- search approved property projections;
- compare public listing state with internal status;
- create a CRM follow-up task;
- prepare a viewing brief;
- generate a management summary from existing reports;
- identify stale or incomplete public records;
- prepare a content brief from approved customer questions.

Writing, deleting, submitting financial documents, changing availability, changing contracts, sending external messages, or triggering bulk communication must be separate privileged actions with explicit confirmation.

Assistant controls

- Frappe roles and document permissions are the primary boundary.
- Do not place credentials, payment data, unrelated company data, or unnecessary tenant identity in prompts.
- Treat model output as untrusted data and validate it before document creation or message delivery.
- Record successful, failed, denied, and timed-out tool actions in the audit log.
- Apply time, memory, CPU, recursion, and output limits to code-execution features.
- Require human approval for public claims, financial actions, bulk communication, and destructive actions.

10. Security and privacy

Trust boundaries

1. Public visitor to website or enquiry endpoint.
2. External channel provider to webhook endpoint.
3. n8n to Frappe API.
4. Frappe to payment provider.

5. User to assistant/MCP endpoint.
6. Uploaded document to extraction/OCR process.
7. Public content to assistant context.

Required controls

- HTTPS and restricted CORS for authenticated interfaces.
 - Environment-managed credentials; no tokens, passwords, or full payment payloads in logs or Git.
 - Webhook signature verification, timestamp/replay protection, and idempotent processing.
 - Rate limits and input-size caps on public lead, chat, upload, and webhook routes.
 - Server-side role and document permission checks for every operation.
 - Allowlisted outbound hosts and no arbitrary server-side URL fetching.
 - Redacted structured logs with correlation IDs.
 - Separate staging credentials, test numbers, test payment references, and production credentials.
 - Backup verification, restore drills, error-log retention, and rollback procedure.
 - Data-retention rules for leads, conversations, uploaded documents, and audit records.
 - Consent and opt-out records for marketing communication.
-

11. Delivery plan by vertical slice

Slice 1 — approved inventory projection

- Define stable listing reference and public field allowlist.
- Map Utility Property records to Tina records.
- Implement read-only projection and unpublished behaviour.
- Verify category/detail pages, metadata, and sitemap.

Exit: a reviewed property can be projected without exposing private fields.

Slice 2 — lead capture

- Implement validated public capture method.
- Create/update CRM Lead with UTM, source, consent, owner, SLA, and next action.
- Add deduplication, rate limits, idempotency, and malformed-payload tests.

Exit: one valid enquiry creates one owned CRM record.

Slice 3 — conversation routing

- Configure approved channel profiles, hours, templates, and routing roles.
- Link inbound conversations to CRM and property references.
- Test signatures, retries, opt-out, attachment limits, and provider failure.

Exit: an inbound enquiry retains context and a responsible owner.

Slice 4 — contract and billing

- Map qualified Deal to Utility Service Request.

- Configure unit selection, deposit, contract, invoice, meter, tariff, and adjustment gates.
- Reconcile Sales Order, Sales Invoice, and Payment Entry using fixtures.
- Test cancellation, amendment, overdue, and renewal paths.

Exit: a test tenancy completes the agreed contract-to-payment path without duplication.

Slice 5 — attribution

- Define event names, UTM persistence, consent, and expiry.
- Join source to CRM Lead, Deal, viewing, contract, invoice, and payment reports.
- Test missing, conflicting, and expired attribution.

Exit: management can compare source-to-qualified-demand and source-to-revenue.

Slice 6 — assistant tools and skills

- Add Eens read-only tools and skills through supported extension points.
- Configure categories and role access.
- Add prompt templates with bounded arguments.
- Test permission denial, audit entries, timeout, stale-data warnings, and confirmation gates.

Exit: approved users receive source-grounded, permission-correct answers.

12. Observability and recovery

The team must be able to answer:

1. Did the enquiry arrive and where did it come from?
2. Was one CRM Lead created or updated, and who owns the next action?
3. Did the conversation, viewing, service request, contract, invoice, or payment event succeed?
4. Which dependency failed: website, n8n, channel provider, Frappe, or payment provider?
5. Did an assistant result use the right source record and permission scope?

Required signals:

- structured events with `eventName`, `correlationId`, `source`, `status`, `durationMs`, and bounded error codes;
- rate, error rate, and p95/p99 duration for public endpoints and dependencies;
- n8n execution age and failed-workflow counts;
- Frappe Error Log and Assistant Audit Log retention with a review owner;
- alerts for lead capture, webhook, invoice generation, payment reconciliation, and public build failures;
- a runbook for every alert with first query, owner, fallback, and escalation.

Every integration needs a manual completion path. A provider outage must not require duplicate records or direct database edits.

13. Acceptance matrix

Area	Minimum test
Public inventory	A non-approved unit cannot appear in a public route or sitemap
Freshness	A changed availability record is detected and routed for review
Lead capture	A valid enquiry creates one CRM Lead with source, consent, owner, SLA, and next action
Deduplication	Replaying an event does not create a duplicate lead, message, invoice, or payment
Conversation	An inbound message links to the correct profile and business record
Contract	Deposit and contract gates prevent invalid invoice creation
Meter billing	Fixture readings produce expected consumption and tariff amount
Finance	Invoice and payment references reconcile to the correct customer and contract
Security	Unauthenticated, unauthorized, malformed, replayed, and oversized requests fail safely
AI read path	Answers respect user permissions and identify the source report or record
AI write path	A write action requires the intended role and explicit confirmation
SEO	Canonical, title, description, sitemap, headings, and visible facts are correct
Performance	Public pages build statically without live ERP or assistant dependency
Recovery	A failed workflow can be retried or manually completed without duplication

14. Decisions required from engineering and Eens

1. Which operational record is authoritative for availability during transition?
2. What stable listing reference maps Utility Property to Tina content?
3. Which public fields and status values are allowed in the projection?
4. Which channel profiles, templates, hours, and consent language are approved?
5. Which payment provider and settlement account are approved first?
6. Which roles may publish availability, submit invoices, send campaigns, and use assistant write tools?
7. What are the retention periods for leads, conversations, files, and audit entries?
8. Who owns staging data, production credentials, backups, alerts, and incident escalation?
9. Which KPIs and event names are accepted as the reporting contract?
10. Who signs each slice's acceptance checklist?

15. Source basis

Local system evidence

The proposal is based on the live Bench and checked source, including:

- Utility Billing hooks and documentation;
- Utility Billing records: `Utility Property`, `Utility Service Request`, `Contract Utility Property Item`, `Meter Reading`, `Utility Bill Structure`, `Billing Adjustment Rule`, and `Utility Billing Settings`;
- CRM records: `CRM Lead`, `CRM Deal`, `CRM Task`, lead source/status, territory, organisation, call log, notes, and settings;
- Assistant Core records: settings, audit log, skills, tool configuration, plugin configuration, prompt templates, categories, and dashboards;
- communication records: profiles, messages, topics, templates, notification records, integrations, and document links;
- `apps/eens_app/eens_app/hooks.py`, which currently has no active Eens business DocTypes or hooks;
- `apps/eens_app/frontend/src/content/property/*.mdx`, `tina/collections/property.ts`, `src/lib/data.ts`, `src/components/BaseHead.astro`, and `astro.config.mjs`.

First-party references

- Frappe REST API: <<https://docs.frappe.io/framework/user/en/api/rest>>
- Frappe hooks: <<https://docs.frappe.io/framework/user/en/python-api/hooks>>
- Frappe users and permissions: <<https://docs.frappe.io/framework/user/en/basics/users-and-permissions>>
- Frappe DocTypes and controllers: <<https://docs.frappe.io/framework/user/en/basics/doctypes/controllers>>
- ERPNext documentation: <<https://docs.erpnext.com/>>
- ERPNext Sales Invoice: <<https://docs.erpnext.com/docs/user/manual/en/sales-invoice>>
- Astro configuration: <<https://docs.astro.build/en/reference/configuration-reference/>>
- Astro routing: <<https://docs.astro.build/en/guides/routing/>>
- Astro sitemap integration: <<https://docs.astro.build/en/guides/integrations-guide/sitemap/>>
- TinaCMS collections: <<https://tina.io/docs/reference/collections>>
- TinaCMS content API: <<https://tina.io/docs/features/data-fetching>>
- Google structured data: <<https://developers.google.com/search/docs/appearance/structured-data/intro-structured-data>>
- Google sitemaps: <<https://developers.google.com/search/docs/crawling-indexing/sitemaps/overview>>
- Google canonical URLs: <<https://developers.google.com/search/docs/crawling-indexing/consolidate-duplicate-urls>>
- Google structured-data policies: <<https://developers.google.com/search/docs/appearance/structured-data/sd-policies>>
- WhatsApp Business Platform: <<https://developers.facebook.com/docs/whatsapp/cloud-api/>>

Document boundary

This is the technical implementation proposal. It intentionally excludes the sales narrative, commercial persuasion, and client-facing investment explanation. Those belong in:

`eens-executive-sales-proposal.md`